Isomer (Localization) Decoys – Library Construction Plan

Target-decoy FDR for phospho site localization, built by permuting fragment m/z

Pioneer.jl – prepared for N. Wamsley

2026-07-03

1. Goal

Give localization the same machinery identification has: a **decoy null**. An *isomer decoy* is the same precursor with a phosphate moved to a **known-wrong** residue. It is co-isolated with the real isomers (same precursor m/z), competes for deconvolution weight at the same scans, and – because it is wrong by construction – its score distribution both **calibrates FLR** and provides the **negative class** for a learned localization discriminant. The residual errors Phase A can't catch are all adjacent-site (e.g. `AALDHSCSPSK` S6-vs-true-S8 at confidence 0.962); isomer decoys are built to look exactly like those, so a model trained target-vs-decoy can learn to down-weight them.

This plan is specifically **how to add these decoys to the library**.

2. The core problem and the construction

Prosit cannot predict our decoys. A decoy places the phospho on a residue that can't be phosphorylated (non-S/T/Y), which is outside Prosit-PTM's training domain – it would return garbage intensities. So we **do not predict decoys**. Instead we **derive each decoy from a real isomer's already-predicted fragments by permuting only the site-determining ion m/z**, keeping every intensity and the RT unchanged. This is deliberate: identical intensities + identical RT make these **maximally hard** decoys – they differ from the real isomer *only* in the masses of the ions that actually determine the site, which is precisely the localization question.

2.1 The m/z permutation (grounded in `DetailedFrag`)

`DetailedFrag` (src/structs/SpectralLibrary/fragment_types.jl) already carries everything needed: `is_b`, `is_y`, `is_p`, `ion_position` (the ordinal), `frag_charge`, `mz`, `intensity`. For a peptide of length `L`, a fragment **contains sequence position `p`** iff:

- **b-ion** of ordinal `i` (residues `1..i`): `p <= i`
- **y-ion** of ordinal `j` (residues `L-j+1..L`): `p >= L - j + 1`
- **precursor/`is_p` ion**: contains everything -> never site-determining -> never changed

To turn a real isomer with phospho at site `k` into a decoy with phospho at site `d`, for each fragment compute `has_k = contains(f,k)` and `has_d = contains(f,d)`:

$$m/z_{\text{decoy}} = m/z_{\text{real}} + \frac{\Delta_\phi}{z_f} \cdot (\mathbb{1}[has_d] - \mathbb{1}[has_k]), \quad \Delta_\phi = 79.966331,\ z_f = \texttt{frag\_charge}$$

i.e. **remove** the phospho mass from fragments that contained `k` but not `d`, **add** it to those that contain `d` but not `k`, leave the rest untouched. Intensity, `ion_type`, `ion_position`, `frag_charge`, `rank` are all copied verbatim. That single pass produces the decoy's fragment list.

2.2 Which real isomer is the base, and multi-phospho

- Each decoy is derived from **one specific real isomer** (the one whose site it moves), so decoy and real form a natural training **pair**. For a mono-phospho isomer at `k`, the decoy moves `k -> d`.
- For a **multi**-phospho isomer (sites `{k1,k2,…}`), move exactly **one** site to a non-S/T/Y neighbour and keep the rest real (a decoy that is wrong at one site). The permutation in §2.1 is applied only for that one `(k_i -> d_i)` move; fragments are recomputed against the change at that site only. This preserves precursor m/z (phospho count unchanged) so the decoy stays a co-isolated sibling.

2.3 Decoy site selection `d` – make them *hard*

`d` = the **nearest non-S/T/Y residue to `k`**, preferring the immediate neighbours `k-1` / `k+1`. Rationale:

- **Non-S/T/Y => guaranteed wrong** (can't be phosphorylated) and – because S/T/Y are excluded – a decoy site can **never coincide with a real site**, so there is *no decoy contamination* (the clean analogue of a reversed ID-decoy never matching a real sequence).
- **Adjacent => hard negative.** For `d = k±1` the only site-determining ions distinguishing decoy from real are the single `b/y` pair breaking between `k` and `d` – exactly the adjacent-site ambiguity that is our real failure mode.
- If both neighbours are S/T/Y, expand outward to the nearest non-S/T/Y. If the peptide is all S/T/Y (no valid `d`), skip the decoy for that isomer (rare).

3. Identical / near-collinear columns => L2 is mandatory

When the distinguishing `b/y` pair for a `(k,d)` move is **not in the library's top-N fragments** (weak or unpredicted), the decoy's fragment list becomes **bit-identical** to the real isomer's -> identical columns in the deconvolution design matrix. This is expected and frequent (it *is* the ambiguous case). Consequences and handling:

- **A light L2 (ridge) penalty is required, not optional.** Identical/near-collinear columns make the unregularized solve ill-posed (weight split is arbitrary). The reg sweep already showed ridge both stabilises and *sharpens* the sibling split (FLR 2.5%->1.3% at matched coverage); with decoys it becomes load-bearing. Wire a small default `optimization.deconvolution.{lambda,reg_type}` = `{~1.0, l2}` into the localization path.
- **Optional exact-dedup for training:** a decoy bit-identical to its base carries no discriminating information (identical features) – drop it from the *training* set. But keep the knowledge: an isomer whose only decoy is identical is *intrinsically unlocalizable* from this library -> flag the group ambiguous rather than pretending confidence.

4. Symmetry, flags, and what trains on what

Two independent decoy axes – keep them orthogonal:

is_decoy (ID)	is_loc_decoy (localization)	meaning	used for
F	F	real target isomer	localized; ID-target; **training (label 0)**
F	T	target isomer-decoy	**training (label 1)**; FLR null
T	F	real ID-decoy isomer	ID FDR (existing)
T	T	ID-decoy's isomer-decoy	deconvolution symmetry only

- **Build isomer decoys for the ID-decoy sequences too** (`(T,T)`), not just targets. Otherwise target groups carry extra collinear columns that decoy groups don't, biasing the per-scan weight split and hence the ID target/decoy competition. Symmetry keeps ID FDR fair.
- **Exclude `is_loc_decoy=T` from ID FDR** – they are real sequences at real m/z and would otherwise be mis-counted as ID targets. They participate in the deconvolution (to split weight and expose wrong reals) but are removed before ID target/decoy counting and normal ID output.
- **Train the localization model only on `is_decoy=F`** – real targets `(F,F)` vs target isomer-decoys `(F,T)`. The `(T,T)` rows exist purely to balance the matrix. This is the standard semi-supervised setup: targets are a mix of correct/incorrect, decoys are all wrong; the model separates the distributions and the decoy tail calibrates FLR.
- **Provenance columns:** `loc_base_prec_id::UInt32` (decoy -> its base real isomer, for pairing + which site moved) and `loc_decoy_site::UInt8` (the moved-to position `d`).

5. Where this lives in BuildSpecLib

Insert a single post-prediction step; **Koina is never called for decoys.**

```
digest -> variable-mod enumeration (real isomers)      [unchanged]
  -> Koina predict real isomers                          [unchanged]
  -> build_detailed_frags_from_raw  -> DetailedFrag[]      [unchanged; has ion_position/is_b/is_y
  -> generate_isomer_decoys!  ← NEW                       [derive decoy precursors + permuted Det
  -> _fill_compact_from_raw! / compaction                [unchanged; decoys flow through like rea
  -> buildPionLib                                         [unchanged + new flag columns]
```

`generate_isomer_decoys!` (new, ~one function): for every precursor with >=1 Unimod:21, and for the chosen movable site, (1) pick `d` (§2.3), (2) synthesize the decoy precursor row – same sequence/charge/mz, `structural_mods` with that Unimod:21 rewritten to `(d, <residue@d>, Unimod:21)`, `is_loc_decoy=true`, `is_decoy` inherited, `loc_base_prec_id`, `loc_decoy_site`, a fresh `precursor_idx` – and (3) emit its `DetailedFrag[]` via the §2.1 permutation of the base's frags (new `prec_id`). Append to the precursor + fragment tables; everything downstream (compaction, index build) is unchanged. It needs only peptide length `L` (from `sequence`) and the base site `k` (parsed from `structural_mods`) – both already present.

Cost knob: one decoy per real isomer roughly doubles the phospho-isomer precursor + fragment count. Expose `build.localization_decoys.{enabled, per_site_cap}`; for pure FLR *calibration* a fractional sample of reals suffices, but 1:1 is cleanest for *training* balance. (A future optimization is to generate decoys **on the fly at search time** for identified groups only – avoiding library bloat – since a decoy is just an m/z permutation of a real isomer already in the index. Build-side first; it is simpler and matches "add decoys to the library".)

6. Search-side use (context; separate plan)

Decoys ride the normal fragment index and deconvolution, so per scan they compete for weight against the real isomers (with the L2 default). Then, per identified isomer group:

1. **Calibration (first, no model):** at each `iso_weight_fraction` cutoff, the fraction of *winning* placements that are `is_loc_decoy` estimates FLR -> a **calibrated localization q-value** replacing the ad-hoc 0.75. Validates the decoys behave before any model.
2. **Learned discriminant (then):** LightGBM, label = `is_loc_decoy`, on per-placement features – `iso_weight_fraction`, absolute weight, and **site-determining-ion evidence** (count / summed intensity / fraction-of-predicted of the ions that uniquely separate this placement from its siblings). Output = localization confidence; decoy tail = its FLR.

7. Validation

- **Decoys must behave as a null:** on the standards, decoys should win weight far less often than real isomers, and the **decoy-estimated FLR must track the ground-truth FLR** we already have (2.5% @ 0.75 no-reg). If decoy-FLR ~= true-FLR, calibration is real.
- **No-harm on ID:** confirm adding loc-decoys to the deconvolution (with L2) does not materially cut real-isomer ID sensitivity (weight-splitting risk) – compare ID counts vs the Phase-A run.
- **Hard-negative check:** decoys placed adjacent should concentrate near the real-isomer score boundary (not pile up at score 0); if they're trivially separable the placement is too easy.

8. Risks / open questions

- **ID-sensitivity vs competition** – loc-decoys in the *main* deconvolution split weight with real isomers; L2 mitigates but quantify the ID cost. Alternative: a separate localization-only deconvolution pass over identified groups (no main-search impact) – heavier architecture, deferred.
- **Library/runtime bloat** – ~2x phospho precursors + fragments; gate + cap; on-the-fly generation is the eventual fix.
- **Residue label in `structural_mods`** – a decoy writes `(d, <non-STY>, Unimod:21)`; confirm no downstream code rejects a Unimod:21 on a non-S/T/Y residue (it shouldn't – matching is by m/z, not residue identity – but verify parse/validate paths).
- **Exact-duplicate accounting** – decide dedup-for-training vs keep-for-ambiguity-flagging (§3).
- **Multi-phospho decoy choice** – moving one site of several; which site, and whether to emit one decoy per movable site (more nulls, more columns) or one per isomer.

9. Phased implementation

1. **`generate_isomer_decoys!` + flags** – the §2/§5 build step; add `is_loc_decoy`, `loc_base_prec_id`, `loc_decoy_site` columns; config gate + cap. Unit-test the m/z permutation against a hand-worked b/y example (site-determining ions shift by ±Delta/z, others fixed).
2. **Build a standards decoy library**, re-search with L2, verify decoys behave as a null (§7) and that decoy-FLR tracks ground-truth FLR.
3. **ID-FDR exclusion + no-harm check** – ensure `is_loc_decoy` rows are dropped from ID counting and ID output; confirm ID sensitivity held.
4. **Calibration** – localization q-value from the decoy tail; compare its cutoff to the tau=0.75 rule.

5. **Learned discriminant** – target vs target-loc-decoy LightGBM on site-ion features; measure FLR vs coverage against Phase A and the paper.